

scheduling_algorithms:

```
#include <stdio.h>

int main()
{
    int n, i, j;
    int bt[20], priority[20], wt[20], tat[20];
    int temp;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time and priority:\n");

    for (i = 0; i < n; i++)
    {
        printf("P%d Burst Time: ", i + 1);
        scanf("%d", &bt[i]);

        printf("P%d Priority: ", i + 1);
        scanf("%d", &priority[i]);
    }

    // Sort according to priority
    for (i = 0; i < n - 1; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            if (priority[i] > priority[j])
            {
                temp = priority[i];
                priority[i] = priority[j];
                priority[j] = temp;

                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }
}
```

```

        temp = bt[i];
        bt[i] = bt[j];
        bt[j] = temp;
    }
}

// Waiting time
wt[0] = 0;

for (i = 1; i < n; i++)
{
    wt[i] = wt[i - 1] + bt[i - 1];
}

// Turnaround time
for (i = 0; i < n; i++)
{
    tat[i] = wt[i] + bt[i];
}

printf("\nProcess\tBT\tPriority\tWT\tTAT\n");

for (i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t\t%d\t%d\n",
        i + 1, bt[i], priority[i], wt[i], tat[i]);
}

return 0;
}
|

```

```

#include <stdio.h>

int main()
{
    int n, i;
    int bt[20], rem_bt[20];
    int wt[20], tat[20];
    int quantum;
    int time = 0;
    int done;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time:\n");

    for (i = 0; i < n; i++)
    {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);

        rem_bt[i] = bt[i];
        wt[i] = 0;
    }

    printf("Enter time quantum: ");
    scanf("%d", &quantum);

    // Round Robin
    while (1)
    {
        done = 1;

        for (i = 0; i < n; i++)
        {
            if (rem_bt[i] > 0)
            {

```

```

        done = 0;

        if (rem_bt[i] > quantum)
        {
            time = time + quantum;
            rem_bt[i] = rem_bt[i] - quantum;
        }
        else
        {
            time = time + rem_bt[i];
            wt[i] = time - bt[i];
            rem_bt[i] = 0;
        }
    }

    if (done == 1)
        break;
}

// Turnaround time
for (i = 0; i < n; i++)
{
    tat[i] = wt[i] + bt[i];
}

printf("\nProcess\tBT\tWT\tTAT\n");

for (i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\n",
           i + 1, bt[i], wt[i], tat[i]);
}

return 0;
}

```

```

bhanu@DESKTOP-DE32HQ8:~$ mkdir -p ~/scheduling_algorithms
cd ~/scheduling_algorithms
bhanu@DESKTOP-DE32HQ8:~/scheduling_algorithms$ pwd
/home/bhanu/scheduling_algorithms
bhanu@DESKTOP-DE32HQ8:~/scheduling_algorithms$ ls
bhanu@DESKTOP-DE32HQ8:~/scheduling_algorithms$ pwd
/home/bhanu/scheduling_algorithms
bhanu@DESKTOP-DE32HQ8:~/scheduling_algorithms$ nano priority.c
bhanu@DESKTOP-DE32HQ8:~/scheduling_algorithms$ gcc priority.c -o priority
bhanu@DESKTOP-DE32HQ8:~/scheduling_algorithms$ ./priority

```

```

bhanu@DESKTOP-DE32HQ8:~/scheduling_algorithms$ ./priority
Enter number of processes: 3
Enter burst time and priority:
P1 Burst Time: 5
P1 Priority: 2
P2 Burst Time: 3
P2 Priority: 1
P3 Burst Time: 8
P3 Priority: 3

Process BT      Priority      WT      TAT
P1      5      2      3      8
P2      3      1      3      6
P3      8      3      8      16
bhanu@DESKTOP-DE32HQ8:~/scheduling_algorithms$ nano round_robin.c
bhanu@DESKTOP-DE32HQ8:~/scheduling_algorithms$ gcc round_robin.c -o round_robin
bhanu@DESKTOP-DE32HQ8:~/scheduling_algorithms$ ./round_robin
Enter number of processes: 3
Enter burst time:
P1: 5
P2: 3
P3: 8
Enter time quantum: 2

Process BT      WT      TAT
P1      5      7      12
P2      3      6      9
P3      8      8      16
bhanu@DESKTOP-DE32HQ8:~/scheduling_algorithms$ |

```